

ТЕХНИЧЕСКОЕ ЗАДАНИЕ

Кошельки и балансы

Wallet Service · Monitor Service · Sweep Scheduler

Параметр	Значение
Версия	1.0
Статус	Черновик
Компонент	Платёжная платформа — кошельки и балансы трейдеров
Внешние зависимости	HashiCorp Vault OSS · PostgreSQL · Redis · TronGrid API
Модули в документе	Wallet Service, Monitor Service, Sweep Scheduler

Все три сервиса (Wallet Service, Monitor Service, Sweep Scheduler) пишутся программистом с нуля.

HashiCorp Vault — self-hosted open-source, без регистрации.

TronGrid API — публичный API, нужна бесплатная регистрация на trongrid.io.

Модуль 1

Wallet Service

Генерация уникальных депозитных адресов трейдеров · BIP44 · HashiCorp Vault

1. Назначение сервиса

Wallet Service — внутренний микросервис, который программист создаёт с нуля. При регистрации нового трейдера сервис генерирует уникальный TRON-адрес по стандарту BIP44 и навсегда закрепляет его за этим трейдером. Адрес не меняется и не переиспользуется.

Сервис не имеет публичных эндпоинтов — вызывается только внутри инфраструктуры при создании трейдера.

2. HashiCorp Vault

HashiCorp Vault — open-source программа (self-hosted), которую программист разворачивает на сервере. Регистрации не требуется. Бесплатная OSS-версия закрывает все нужды платформы. Vault выполняет три роли одновременно:

Роль	Что хранит / делает	Vault Engine
Хранение мастер-seed	Записывается один раз при инициализации. Бэкенд его не читает в открытом виде.	KV v2
Счётчик derivation index	Атомарный инкремент — гарантирует уникальность индекса для каждого трейдера.	KV v2
Хранение частных ключей	Каждый ключ по пути secret/wallets/{trader_id}. Wallet Service только создаёт, не читает.	KV v2
Подписание sweep-транзакций	Sweep Scheduler передаёт raw bytes; Vault подписывает и возвращает готовую tx. Ключ не покидает Vault.	Transit

SDK для обращения к Vault:

Язык	SDK	Установка
Go	github.com/hashicorp/vault/api	go get github.com/hashicorp/vault/api
Python	hvac	pip install hvac
Node.js	node-vault	npm install node-vault

Переменные окружения Wallet Service:

```
VAULT_ADDR=http://vault:8200
VAULT_ROLE_ID=<role_id>
```

```
VAULT_SECRET_ID=<secret_id>
DATABASE_URL=postgresql://user:pass@postgres/platform
```

3. TronGrid API

TronGrid — официальный публичный API сети TRON. Используется Monitor Service для мониторинга и Sweep Scheduler для broadcast транзакций.

Параметр	Значение
Базовый URL	https://api.trongrid.io
Регистрация	https://www.trongrid.io — бесплатно, нужен email
Заголовок запроса	TRON-PRO-API-KEY: {api_key}
Для продакшна	Платный Pro-план или собственная TRON full node

4. Поток генерации адреса

Шаг	Кто	Действие	Детали
1	Admin API	POST /api/traders (регистрация)	Бэкенд получает запрос на создание трейдера
2	Wallet Service	Запрос к Vault: атомарный инкремент derivation index	Читает current_index, пишет current_index+1 в secret/wallet_counter
3	Wallet Service	Чтение master seed из Vault	GET secret/data/master_seed — поле seed (mnemonic)
4	Wallet Service	Деривация ключа по BIP44	Путь: m/44'/195'/0'/0/{index} — coin type 195 = TRON
5	Wallet Service	Вычисление TRON-адреса	ECDSA secp256k1 → keccak256(pubkey) → base58check, префикс 0x41
6	Wallet Service	Сохранение приватного ключа в Vault	PUT secret/data/wallets/{trader_id} → {private_key, address, index}
7	Wallet Service	Сохранение адреса в PostgreSQL	INSERT INTO trader_wallets(trader_id, address, derivation_index)
8	Admin API	Ответ клиенту	trader_id + deposit_address

ВАЖНО: Шаги 2 и 3 должны быть защищены распределённой блокировкой (Redis SETNX), чтобы два одновременных запроса не получили одинаковый derivation index.

5. BIP44 деривация

Путь деривации для TRON:

```
m / 44' / 195' / 0' / 0 / index
```

Уровень	Значение	Пояснение
<code>coin_type</code>	195'	TRON — зарегистрирован в SLIP-0044 (hardened)
<code>account</code>	0'	Первый аккаунт, фиксировано (hardened)
<code>change</code>	0	External chain, фиксировано
<code>index</code>	N	Уникальный порядковый номер трейдера из Vault

Рекомендуемые библиотеки:

Язык	Библиотека	Установка
Go	go-bip32 + btcd/btcec	go get github.com/tyler-smith/go-bip32
Python	tronpy + hdwallet	pip install tronpy hdwallet
Node.js	tronweb + bip39 + hdkey	npm install tronweb bip39 hdkey

6. Схема БД — Wallet Service

Таблица: `trader_wallets`

Колонка	Тип	Ограничения	Описание
<code>id</code>	uuid	PK, default <code>gen_random_uuid()</code>	Первичный ключ
<code>trader_id</code>	uuid	FK → <code>traders.id</code> , UNIQUE, NOT NULL	Один адрес на одного трейдера
<code>address</code>	varchar(42)	NOT NULL, UNIQUE	TRON-адрес в base58check (начинается с T)
<code>derivation_index</code>	integer	NOT NULL, UNIQUE	Порядковый номер из Vault
<code>vault_path</code>	varchar(255)	NOT NULL	<code>secret/data/wallets/{trader_id}</code>
<code>created_at</code>	timestampz	NOT NULL, default <code>now()</code>	Время генерации

7. API эндпоинты Wallet Service

Все эндпоинты — только внутренние (inter-service). Недоступны из интернета.

POST `/internal/wallets/generate`

Генерация депозитного адреса для трейдера.

```
Body: { "trader_id": "uuid" }
201: { "trader_id": "...", "address": "Txxx...", "derivation_index": 42 }
409: адрес для этого trader_id уже существует
```

GET /internal/wallets/{trader_id}

Получение адреса трейдера.

```
200: { "trader_id": "...", "address": "Txxx...", "derivation_index": 42,
      "created_at": "..." }
```

8. Структура секретов в Vault

Vault path	Содержимое	Кто имеет доступ
secret/data/master_seed	{ "seed": "24 слова" }	Wallet Service — только read
secret/data/wallet_counter	{ "current_index": 42 }	Wallet Service — read + update
secret/data/wallets/{trader_id}	{ "private_key": "hex", "address": "Txxx" }	Sweep Scheduler — read (для подписания)

Vault Policy — Wallet Service:

```
path "secret/data/master_seed" { capabilities = ["read"] }
path "secret/data/wallet_counter" { capabilities = ["read", "update", "create"] }
path "secret/data/wallets/*" { capabilities = ["create", "update"] }
```

Wallet Service НЕ может читать приватные ключи — только создавать. Чтение ключей разрешено исключительно Sweep Scheduler через отдельную policy.

9. Первоначальная инициализация (one-time setup)

Шаг	Действие
1. Генерация seed	ОФЛАЙН: <code>python -c "from mnemonic import Mnemonic; print(Mnemonic('english').generate(256))"</code>
2. Запись seed	<code>vault kv put secret/master_seed seed='24 слова'</code>
3. Счётчик	<code>vault kv put secret/wallet_counter current_index=0</code>
4. AppRole	<code>vault auth enable approle && vault write auth/approle/role/wallet-service token_policies=wallet-policy</code>
5. Получить role_id	<code>vault read auth/approle/role/wallet-service/role-id</code>
6. Получить secret_id	<code>vault write -f auth/approle/role/wallet-service/secret-id</code>
7. В переменные	VAULT_ROLE_ID и VAULT_SECRET_ID — через .env или secrets manager

Мастер-seed генерируется ОФЛАЙН. После записи в Vault исходный файл физически уничтожается. Никогда не хранить seed в коде, .env или git.

Модуль 2

Monitor Service

Мониторинг входящих USDT-транзакций · Зачисление виртуального баланса · Realtime Push

1. Назначение сервиса

Monitor Service — внутренний сервис, который непрерывно отслеживает входящие USDT (TRC-20) транзакции на все депозитные адреса трейдеров. После подтверждения транзакции блокчейном сервис атомарно записывает её в БД, зачисляет виртуальный баланс трейдеру и отправляет realtime-уведомление в кабинет.

Виртуальный баланс — это цифра в БД, которую видит трейдер в кабинете.
Реальный USDT остаётся на депозитном адресе до момента свипа (Sweep Scheduler забирает его на Tangem).
Monitor Service только фиксирует факт поступления и обновляет баланс в БД. Физическое движение USDT — задача Sweep Scheduler.

2. Как работает мониторинг

2.1 Polling через TronGrid API

Monitor Service каждые 5–10 секунд опрашивает TronGrid API. Вместо поллинга каждого адреса по отдельности — один запрос к событиям контракта USDT (TRC-20), фильтрация по получателям локально. Это в разы эффективнее и не упирается в rate limit даже при большом числе трейдеров.

Параметр	Значение
Контракт USDT TRC-20	TR7NHqjeKQxGTCi8q8ZY4pL8otSzgijLj6t
Метод TronGrid	GET /v1/contracts/{contract}/events?event_name=Transfer
Интервал поллинга	5–10 секунд
Подтверждения блоков	19 (стандарт безопасности для TRC-20 USDT)
Дедупликация	Уникальный индекс на поле tx_hash в таблице deposits

2.2 Полный поток обработки транзакции

Шаг	Действие	Детали
1	Запрос событий USDT-контракта к TronGrid API	GET /v1/contracts/TR7NHqeKQxGTCi8q8ZY4pL8otSzgJLj6t/events
2	Фильтрация по нашим адресам	Из ответа оставить только Transfer, где поле to совпадает с адресом в trader_wallets
3	Проверка на дубль по tx_hash	SELECT COUNT(*) FROM deposits WHERE tx_hash = \$1 — если > 0, пропустить
4	Ожидание 19 подтверждений блоков	Повторные запросы с интервалом 5–10 сек до достижения confirmations >= 19
5	Атомарная запись в БД (транзакция)	INSERT INTO deposits + UPDATE trader_balances — в одной SQL-транзакции
6	Отправка события в Realtime Push	Публикация события в Redis Pub/Sub канал трейдера: { trader_id, amount, tx_hash }
7	Публикация события для Sweep Scheduler	Публикация в Redis канал sweep_check: { trader_id, address, on_chain_balance }

Шаги 3 и 5 защищают от двойного зачисления. Уникальный индекс на tx_hash в БД — финальная страховка даже при баге в логике приложения.

3. Схема БД — Monitor Service

Таблица: deposits

Колонка	Тип	Ограничения	Описание
id	uuid	PK, default gen_random_uuid()	Первичный ключ
trader_id	uuid	FK → traders.id, NOT NULL	Трейдер-получатель
address	varchar(42)	NOT NULL	Депозитный адрес получателя
tx_hash	varchar(128)	NOT NULL, UNIQUE	Hash транзакции в блокчейне — защита от дублей
amount_usdt	numeric(20,6)	NOT NULL	Сумма USDT в транзакции
confirmations	integer	NOT NULL, default 0	Количество подтверждений на момент записи
block_number	bigint	NOT NULL	Номер блока транзакции
status	varchar(20)	NOT NULL, default 'confirmed'	confirmed / failed
created_at	timestampz	NOT NULL, default now()	Время записи в БД

Таблица: trader_balances

Колонка	Тип	Ограничения	Описание
trader_id	uuid	PK, FK → traders.id	Один баланс на трейдера

Колонка	Тип	Ограничения	Описание
balance_usdt	numeric(20,6)	NOT NULL, default 0	Текущий виртуальный баланс (зачислено минус выплачено)
total_deposited	numeric(20,6)	NOT NULL, default 0	Сумма всех зачислений за всё время
updated_at	timestampz	NOT NULL	Время последнего обновления

Обновление баланса — атомарная операция в одной SQL-транзакции:

```
BEGIN;
INSERT INTO deposits (trader_id, address, tx_hash, amount_usdt, confirmations,
block_number)
VALUES ($1, $2, $3, $4, $5, $6);
UPDATE trader_balances
SET balance_usdt      = balance_usdt + $4,
    total_deposited   = total_deposited + $4,
    updated_at        = now()
WHERE trader_id = $1;
COMMIT;
```

4. Realtime Push

После успешной записи в БД Monitor Service публикует событие в Redis Pub/Sub. Отдельный Realtime Push сервис (WebSocket / SSE) подписан на эти каналы и мгновенно доставляет уведомление в браузер трейдера.

Параметр	Значение
Транспорт	Redis Pub/Sub
Канал	trader:{trader_id}:events
Payload события	{ "type": "deposit", "amount": "100.00", "tx_hash": "...", "timestamp": "..." }
Frontend-протокол	WebSocket или Server-Sent Events (SSE) — на выбор команды

5. Конфигурация Monitor Service

Переменная окружения	Описание
TRONGRID_API_KEY	API-ключ TronGrid (получить на trongrid.io)
TRONGRID_BASE_URL	https://api.trongrid.io
USDT_CONTRACT	TR7NHqjeKQxGTCi8q8ZY4pL8otSzgJLj6t
POLL_INTERVAL_SEC	5 (интервал опроса в секундах)
MIN_CONFIRMATIONS	19
DATABASE_URL	postgresql://user:pass@postgres/platform
REDIS_URL	redis://redis:6379

6. Зависимости Monitor Service

Зависимость	Назначение	Self-hosted?
TronGrid API	Получение событий USDT-контракта	Нет (публичный API)
PostgreSQL	Хранение депозитов и балансов	Да
Redis	Pub/Sub для Realtime Push + флаги дедупликации	Да

Модуль 3

Sweep Scheduler

Автоматический перевод USDT на холодный кошелёк Tangem · Vault-подписание · TRX-стейкинг

1. Назначение сервиса

Sweep Scheduler — внутренний сервис, который периодически проверяет on-chain балансы депозитных адресов трейдеров. Когда баланс адреса достигает порогового значения, сервис через Vault подписывает транзакцию и отправляет USDT с адреса трейдера напрямую на адрес холодного кошелька Tangem. Все операции фиксируются в таблице sweep_log.

Sweep Scheduler работает с реальными on-chain USDT — не с виртуальным балансом из БД.

Виртуальный баланс трейдера в БД НЕ изменяется при свипе — он уже был зачислен Monitor Service при поступлении депозита.

Свип — это исключительно операция по переносу реального USDT с депозитного адреса на Tangem.

2. Триггеры запуска

Sweep Scheduler запускается двумя способами одновременно. Это позволяет не ждать cron-интервала при крупном пополнении и при этом не пропускать адреса, по которым долго не было активности.

Триггер	Как работает	Когда срабатывает
Cron (периодический)	Каждые 5 минут обходит все активные адреса, проверяет on-chain баланс через TronGrid API	Фоновая проверка всех адресов — страховка против пропусков
Event от Monitor Service	Monitor Service после зачисления публикует событие в Redis канал sweep_check. Sweep Scheduler подписан на канал и немедленно проверяет конкретный адрес	Сразу после подтверждённого депозита — без ожидания cron

Оба триггера независимы и не конфликтуют: защита от двойного свипа реализуется через Redis-блокировку (SETNX sweep_lock:{address}) на время выполнения операции.

Параметр	Значение
Cron-расписание	Каждые 5 минут (* / 5 * * * *)

Параметр	Значение
Redis event-канал	sweep_check — публикует Monitor Service после зачисления
Payload события	{ "trader_id": "...", "address": "Txxx...", "on_chain_balance": "1500.00" }
Порог свипа	Настраиваемый, по умолчанию 1000 USDT
Источник баланса	On-chain запрос к TronGrid API (не из БД)
Адрес назначения	Адрес холодного кошелька Tangem (задаётся в конфиге)
Подписание транзакции	HashiCorp Vault Transit Engine — приватный ключ не покидает Vault
Broadcast транзакции	TronGrid API — POST /wallet/broadcasttransaction

3. Полный поток свипа

Шаг	Действие	Детали
1	Триггер: cron или Redis-событие	Cron: проверяет все адреса. Redis (sweep_check): проверяет конкретный адрес немедленно после депозита
2	Захват Redis-блокировки на адрес	SETNX sweep_lock:{address} TTL=60s — если ключ уже есть, другой воркер обрабатывает, пропустить
3	Загрузка адресов из БД (только для cron)	SELECT address, trader_id FROM trader_wallets WHERE active = true
4	Запрос on-chain баланса USDT каждого адреса	GET /v1/accounts/{address}/balances через TronGrid API
5	Проверка порога	IF on_chain_balance_usdt >= SWEEP_THRESHOLD → продолжить свип
6	Расчёт суммы свипа	amount = on_chain_balance_usdt — оставить минимальный TRX-буфер (~5 TRX) для bandwidth
7	Получение приватного ключа из Vault	GET secret/data/wallets/{trader_id} → private_key
8	Формирование raw-транзакции TRC-20 Transfer	from = адрес трейдера, to = TANGEM_ADDRESS, amount = amount
9	Подписание через Vault Transit	Vault подписывает bytes транзакции, возвращает подписанную tx
10	Broadcast транзакции в сеть	POST /wallet/broadcasttransaction в TronGrid API
11	Запись в sweep_log	INSERT: trader_id, from_address, tx_hash, amount, status, created_at
12	Освобождение Redis-блокировки + обновление кэша	Опционально: сбросить кэш баланса адреса в Redis

Шаг 6 — единственный момент, когда приватный ключ используется. Vault возвращает уже подписанную транзакцию, никогда не передавая сам ключ в бэкенд.

4. Оптимизация комиссий — TRX Staking (Energy)

Каждая TRC-20 USDT транзакция стоит примерно 13 TRX (~1–2 USD) или потребляет ~32 000 единиц Energy. При наличии достаточного стейкинга TRX транзакция проходит бесплатно.

Стейкинг TRX — это заморозка TRX на аккаунте в обмен на ресурс Energy. Energy тратится на выполнение смарт-контрактов (включая TRC-20 Transfer), а не на обычные переводы TRX. Одна USDT TRC-20 Transfer = ~32 000 Energy. При 100 свипах в день нужно ~3 200 000 Energy.

Параметр	Значение
Ресурс	Energy (не Bandwidth)
Механизм получения	Stake TRX v2: freezeBalanceV2(amount, ENERGY)
Делегирование	delegateResource(from, to, amount, ENERGY) — делегировать Energy на адреса трейдеров перед свипом
Восстановление Energy	Полностью восстанавливается каждые 24 часа
Расчёт стейка	1 000 000 TRX стейка ≈ 1 000 Energy/сек — точные цифры уточнять на tronscan.org
TRX-буфер на адресах	Держать 5–10 TRX на каждом депозитном адресе на случай нехватки Energy (Bandwidth fallback)

Рекомендуемая схема стейкинга:

Создать отдельный «master» TRX-аккаунт. На нём застейкать TRX и делегировать Energy на депозитные адреса трейдеров непосредственно перед свипом (delegateResource). После завершения свипа делегирование можно отозвать (undelegateResource).

5. Схема БД — Sweep Scheduler

Таблица: sweep_log

Колонка	Тип	Ограничения	Описание
id	uuid	PK, default gen_random_uuid()	Первичный ключ
trader_id	uuid	FK → traders.id, NOT NULL	Трейдер, с чьего адреса выполнен свип
from_address	varchar(42)	NOT NULL	Депозитный адрес трейдера
to_address	varchar(42)	NOT NULL	Адрес Tangem — холодный кошелёк
tx_hash	varchar(128)	UNIQUE	Hash транзакции свипа
amount_usdt	numeric(20,6)	NOT NULL	Сумма USDT в свипе

Колонка	Тип	Ограничения	Описание
fee_trx	numeric(20,6)	default 0	Реальная комиссия в TRX (0 при наличии Energy)
status	varchar(20)	NOT NULL, default 'pending'	pending / broadcast / confirmed / failed
error	text	nullable	Описание ошибки при failed
created_at	timestampz	NOT NULL, default now()	Время создания записи
confirmed_at	timestampz	nullable	Время подтверждения в блокчейне

6. Vault Policy — Sweep Scheduler

Sweep Scheduler имеет права читать приватные ключи трейдеров (для подписания) и использовать Transit Engine:

```
path "secret/data/wallets/*" {
  capabilities = ["read"]
}
path "transit/sign/sweep-key" {
  capabilities = ["update"]
}
```

7. Конфигурация Sweep Scheduler

Переменная окружения	Описание
SWEEP_THRESHOLD_USDT	Пороговое значение для свипа (по умолчанию: 1000)
TANGEM_ADDRESS	Адрес холодного кошелька Tangem — фиксированный, задаётся при деплое
VAULT_ADDR	http://vault:8200
VAULT_ROLE_ID	AppRole role_id для Sweep Scheduler
VAULT_SECRET_ID	AppRole secret_id для Sweep Scheduler
TRONGRID_API_KEY	API-ключ TronGrid
DATABASE_URL	postgresql://user:pass@postgres/platform
REDIS_URL	redis://redis:6379
CRON_SCHEDULE	* / 5 * * * * (каждые 5 минут)

Итог

Что пишет программист, что разворачивается, что подключается снаружи

Компонент	Тип	Регистрация?	Кто отвечает
Wallet Service	Пишется с нуля	Нет	Программист
Monitor Service	Пишется с нуля	Нет	Программист
Sweep Scheduler	Пишется с нуля	Нет	Программист
HashiCorp Vault OSS	Self-hosted, скачать+запустить	Нет	Программист / DevOps
PostgreSQL	Self-hosted	Нет	DevOps
Redis	Self-hosted	Нет	DevOps
TronGrid API	Публичный API	Да (бесплатно, trongrid.io)	Программист
TRX для стейкинга	Купить на бирже	Нет	Владелец платформы

Следующий модуль: Realtime Push — WebSocket/SSE сервис для мгновенных уведомлений в кабинете трейдера.